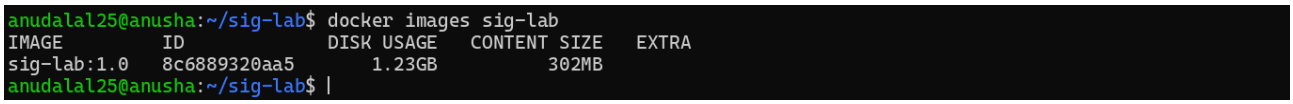
Sardar Patel Institute of Technology - Department of Computer Engineering
Cryptography and Network Security Lab

Experiment 8: Evaluation of Classical and Post-Quantum Digital Signature Algorithms using Docker-Based Client-Server Communication

Screenshots (in order of execution)

Name: Anusha Dalal UID: 2024300032 Date: 16 October 2026

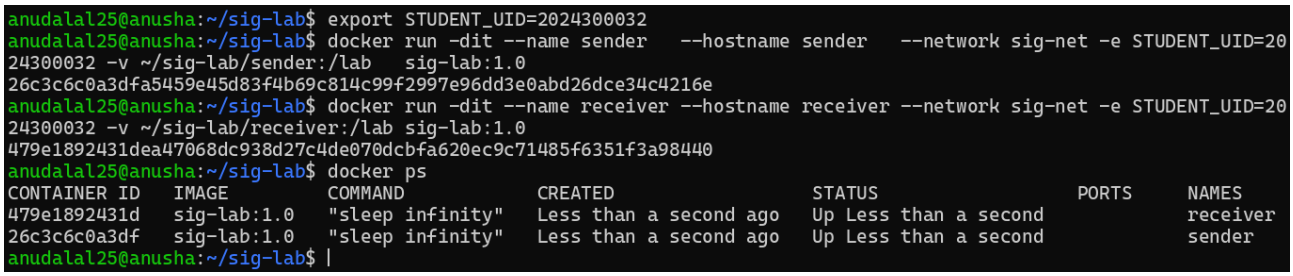
Screenshot 1: Step C4: the image sig-lab:1.0 has been built and is listed by docker images



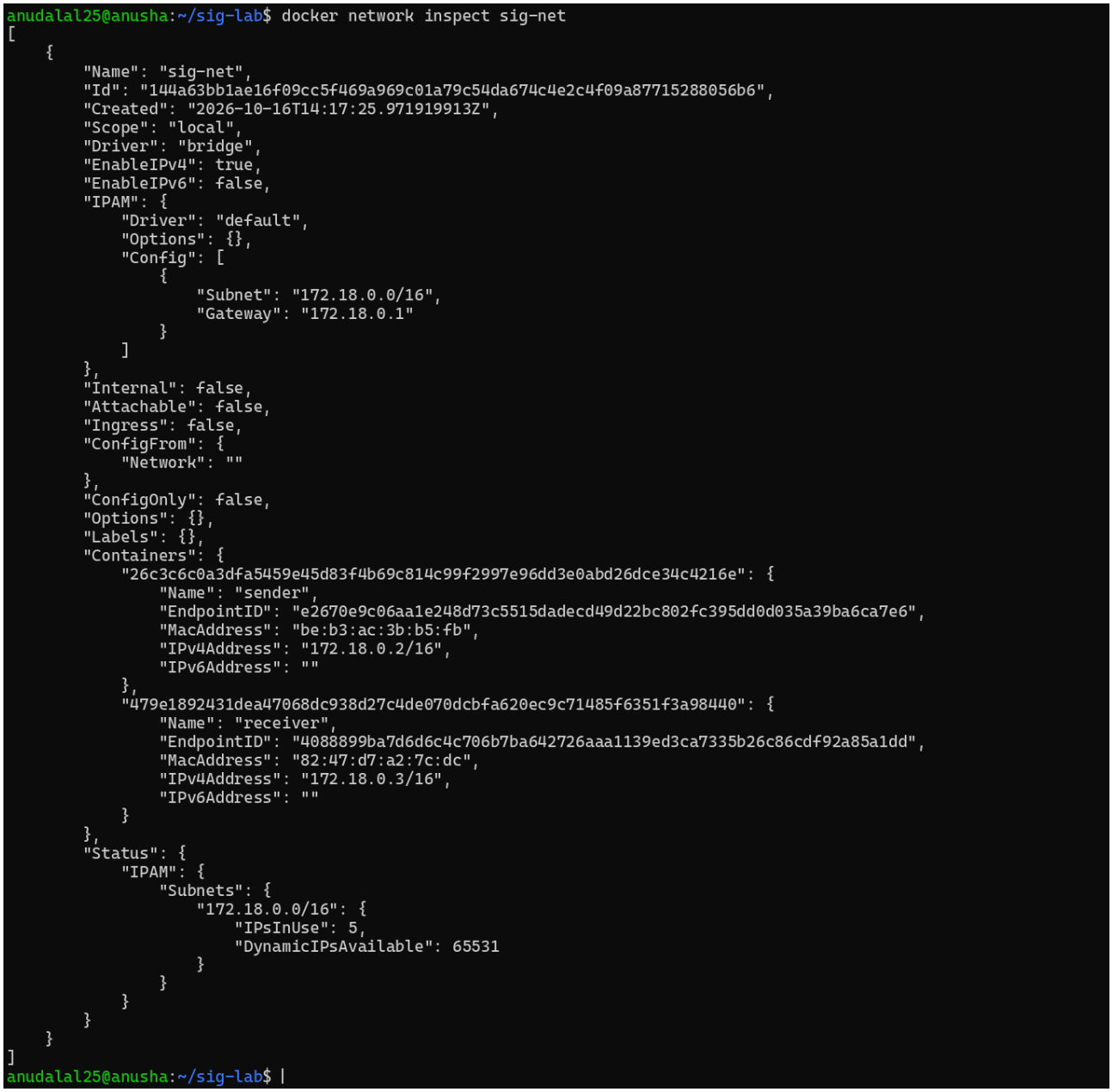
Screenshot 2: Step C5: user-defined bridge network sig-net created and listed (no containers attached yet)

```
anudalal25@anusha:~/sig-lab$ docker network create sig-net
144a63bb1ae16f09cc5f469a969c01a79c54da674c4e2c4f09a87715288056b6
anudalal25@anusha:~/sig-lab$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
f75c0c8e7cd5   bridge    bridge    local
482ed9f0038a   host      host      local
0da95fc264de   none      null      local
144a63bb1ae1   sig-net   bridge    local
anudalal25@anusha:~/sig-lab$ docker network inspect sig-net
[
    {
        "Name": "sig-net",
        "Id": "144a63bb1ae16f09cc5f469a969c01a79c54da674c4e2c4f09a87715288056b6",
        "Created": "2026-10-16T14:17:25.971919913Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {},
        "Labels": {},
        "Containers": {},
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.18.0.0/16": {
                        "IPsInUse": 3,
                        "DynamicIPsAvailable": 65533
                    }
                }
            }
        }
    }
]
anudalal25@anusha:~/sig-lab$
```

Screenshot 3: Step C6: sender and receiver containers started on sig-net with STUDENT_UID; docker ps shows both Up



Screenshot 4: Step C7: docker network inspect sig-net now lists both containers with their IP addresses



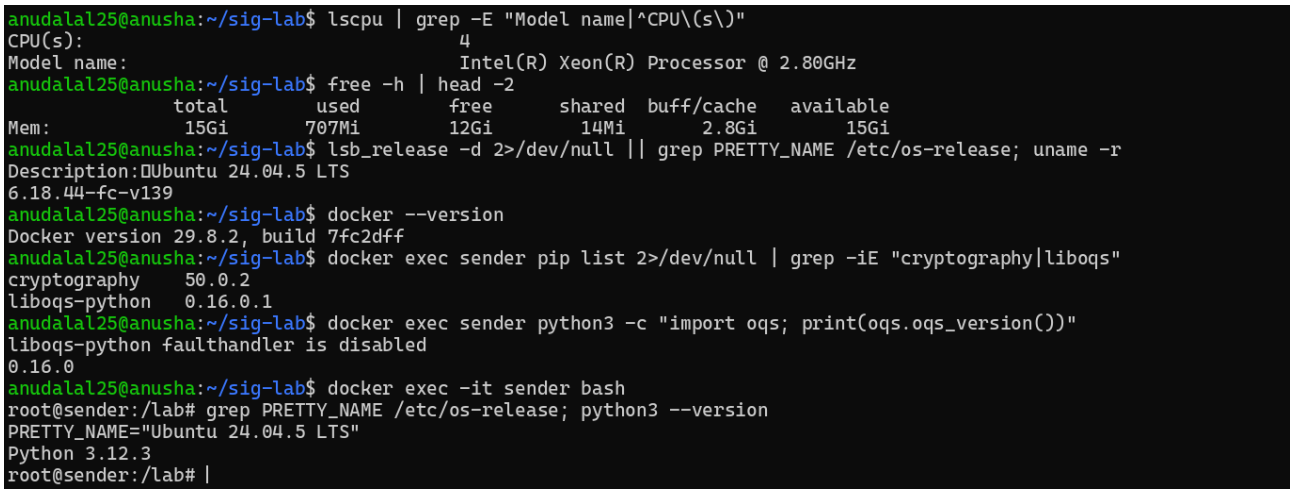
```
anudalal25@anusha:~/sig-lab$ docker network inspect sig-net
[
    {
        "Name": "sig-net",
        "Id": "144a63bb1ae16f09cc5f469a969c01a79c54da674c4e2c4f09a87715288056b6",
        "Created": "2026-10-16T14:17:25.971919913Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {},
        "Labels": {},
        "Containers": {
            "26c3c6c0a3dfa5459e45d83f4b69c814c99f2997e96dd3e0abd26dce34c4216e": {
                "Name": "sender",
                "EndpointID": "e2670e9c06aa1e248d73c5515dadecd49d22bc802fc395dd0d035a39ba6ca7e6",
                "MacAddress": "be:b3:ac:3b:b5:fb",
                "IPv4Address": "172.18.0.2/16",
                "IPv6Address": ""
            },
            "479e1892431dea47068dc938d27c4de070dcbfa620ec9c71485f6351f3a98440": {
                "Name": "receiver",
                "EndpointID": "4088899ba7d6d6c4c706b7ba642726aaa1139ed3ca7335b26c86cdf92a85a1dd",
                "MacAddress": "82:47:d7:a2:7c:dc",
                "IPv4Address": "172.18.0.3/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.18.0.0/16": {
                        "IPsInUse": 5,
                        "DynamicIPsAvailable": 65531
                    }
                }
            }
        }
    }
]
anudalal25@anusha:~/sig-lab$
```

Screenshot 5: Step C7: ping by name from sender to receiver, STUDENT_UID inside the container, liboqs mechanisms and container IPs

```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# ping -c 3 receiver
PING receiver (172.18.0.3) 56(84) bytes of data.
64 bytes from receiver.sig-net (172.18.0.3): icmp_seq=1 ttl=64 time=1.75 ms
64 bytes from receiver.sig-net (172.18.0.3): icmp_seq=2 ttl=64 time=0.073 ms
64 bytes from receiver.sig-net (172.18.0.3): icmp_seq=3 ttl=64 time=0.092 ms

--- receiver ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2029ms
rtt min/avg/max/mdev = 0.073/0.639/1.752/0.787 ms
root@sender:/lab# echo $STUDENT_UID
2024300032
root@sender:/lab# python3 -c "import oqs; print(oqs.get_enabled_sig_mechanisms())"
liboqs-python faulthandler is disabled
('ML-DSA-44', 'ML-DSA-65', 'ML-DSA-87', 'Falcon-512', 'Falcon-1024', 'Falcon-padded-512', 'Falcon-padded-1024', 'MAYO-1', 'MAYO-2', 'MAYO-3', 'MAYO-5', 'cross-rsdp-128-balanced', 'cross-rsdp-128-fast', 'cross-rsdp-128-small', 'cross-rsdp-192-balanced', 'cross-rsdp-192-fast', 'cross-rsdp-192-small', 'cross-rsdp-256-balanced', 'cross-rsdp-256-fast', 'cross-rsdp-256-small', 'cross-rsdpg-128-balanced', 'cross-rsdpg-128-fast', 'cross-rsdpg-128-small', 'cross-rsdpg-192-balanced', 'cross-rsdpg-192-fast', 'cross-rsdpg-192-small', 'cross-rsdpg-256-balanced', 'cross-rsdpg-256-fast', 'cross-rsdpg-256-small', 'OV-Is', 'OV-Ip', 'OV-III', 'OV-V', 'OV-Is-pkc', 'OV-Ip-pkc', 'OV-III-pkc', 'OV-V-pkc', 'OV-Is-pkc-skc', 'OV-Ip-pkc-skc', 'OV-III-pkc-skc', 'OV-V-pkc-skc', 'SNOVA_24_5_4', 'SNOVA_24_5_4_SHAKE', 'SNOVA_24_5_4_esk', 'SNOVA_24_5_4_SHAKE_esk', 'SNOVA_37_17_2', 'SNOVA_25_8_3', 'SNOVA_56_25_2', 'SNOVA_49_11_3', 'SNOVA_37_8_4', 'SNOVA_24_5_5', 'SNOVA_60_10_4', 'SNOVA_29_6_5', 'mqom2_cat1_gf16_fast_r3', 'mqom2_cat1_gf16_fast_r5', 'mqom2_cat1_gf16_short_r3', 'mqom2_cat1_gf16_short_r5', 'mqom2_cat3_gf16_fast_r3', 'mqom2_cat3_gf16_fast_r5', 'mqom2_cat3_gf16_short_r3', 'mqom2_cat3_gf16_short_r5', 'mqom2_cat5_gf16_fast_r3', 'mqom2_cat5_gf16_fast_r5', 'mqom2_cat5_gf16_short_r3', 'mqom2_cat5_gf16_short_r5', 'SLH_DSA_PURE_SHA2_128S', 'SLH_DSA_PURE_SHA2_128F', 'SLH_DSA_PURE_SHA2_192S', 'SLH_DSA_PURE_SHA2_192F', 'SLH_DSA_PURE_SHA2_256S', 'SLH_DSA_PURE_SHA2_256F', 'SLH_DSA_PURE_SHAKE_128S', 'SLH_DSA_PURE_SHAKE_128F', 'SLH_DSA_PURE_SHAKE_192S', 'SLH_DSA_PURE_SHAKE_192F', 'SLH_DSA_PURE_SHAKE_256S', 'SLH_DSA_PURE_SHAKE_256F', 'SLH_DSA_SHA2_224_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_224_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_224_PREHASH_SHA2_192S', 'SLH_DSA_SHA2_224_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_224_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_224_PREHASH_SHA2_256F', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_192S', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_256F', 'SLH_DSA_SHA2_256_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_256_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_256_PREHASH_SHA2_192S', 'SLH_DSA_SHA2_256_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_256_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_256_PREHASH_SHA2_256F', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_192S', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_256F', 'SLH_DSA_SHA2_384_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_384_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_384_PREHASH_SHA2_192S', 'SLH_DSA_SHA2_384_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_384_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_384_PREHASH_SHA2_256F', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_192S', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_256F', 'SLH_DSA_SHA2_512_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_512_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_512_PREHASH_SHA2_192S', 'SLH_DSA_SHA2_512_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_512_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_512_PREHASH_SHA2_256F', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_192S', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_256F', 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_192S', 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_256F', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_192S', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_256F', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_192S', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_256F', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_192S', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_256F', 'SLH_DSA_SHA3_224_PREHASH_SHA2_128S', 'SLH_DSA_SHA3_224_PREHASH_SHA2_128F', 'SLH_DSA_SHA3_224_PREHASH_SHA2_192S', 'SLH_DSA_SHA3_224_PREHASH_SHA2_192F', 'SLH_DSA_SHA3_224_PREHASH_SHA2_256S', 'SLH_DSA_SHA3_224_PREHASH_SHA2_256F', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_128S', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_128F', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_192S', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_192F', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_256S', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_256F', 'SLH_DSA_SHA3_256_PREHASH_SHA2_128S', 'SLH_DSA_SHA3_256_PREHASH_SHA2_128F', 'SLH_DSA_SHA3_256_PREHASH_SHA2_192S', 'SLH_DSA_SHA3_256_PREHASH_SHA2_192F', 'SLH_DSA_SHA3_256_PREHASH_SHA2_256S', 'SLH_DSA_SHA3_256_PREHASH_SHA2_256F', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_128S', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_128F', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_192S', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_192F', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_256S', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_256F', 'SLH_DSA_SHA3_384_PREHASH_SHA2_128S', 'SLH_DSA_SHA3_384_PREHASH_SHA2_128F', 'SLH_DSA_SHA3_384_PREHASH_SHA2_192S', 'SLH_DSA_SHA3_384_PREHASH_SHA2_192F', 'SLH_DSA_SHA3_384_PREHASH_SHA2_256S', 'SLH_DSA_SHA3_384_PREHASH_SHA2_256F', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_128S', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_128F', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_192S', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_192F', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_256S', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_256F', 'SLH_DSA_SHA3_512_PREHASH_SHA2_128S', 'SLH_DSA_SHA3_512_PREHASH_SHA2_128F', 'SLH_DSA_SHA3_512_PREHASH_SHA2_192S', 'SLH_DSA_SHA3_512_PREHASH_SHA2_192F', 'SLH_DSA_SHA3_512_PREHASH_SHA2_256S', 'SLH_DSA_SHA3_512_PREHASH_SHA2_256F', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_128S', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_128F', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_192S', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_192F', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_256S', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_256F', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_128S', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_128F', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_192S', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_192F', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_256S', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_256F', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_128S', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_128F', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_192S', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_192F', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_256S', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_256F', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_128S', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_128F', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_192S', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_192F', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_256S', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_256F', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_128S', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_128F', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_192S', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_192F', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_256S', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_256F')
root@sender:/lab# exit
exit
anudalal25@anusha:~/sig-lab$ docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' sender
172.18.0.2
anudalal25@anusha:~/sig-lab$ docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' receiver
172.18.0.3
anudalal25@anusha:~/sig-lab$
```

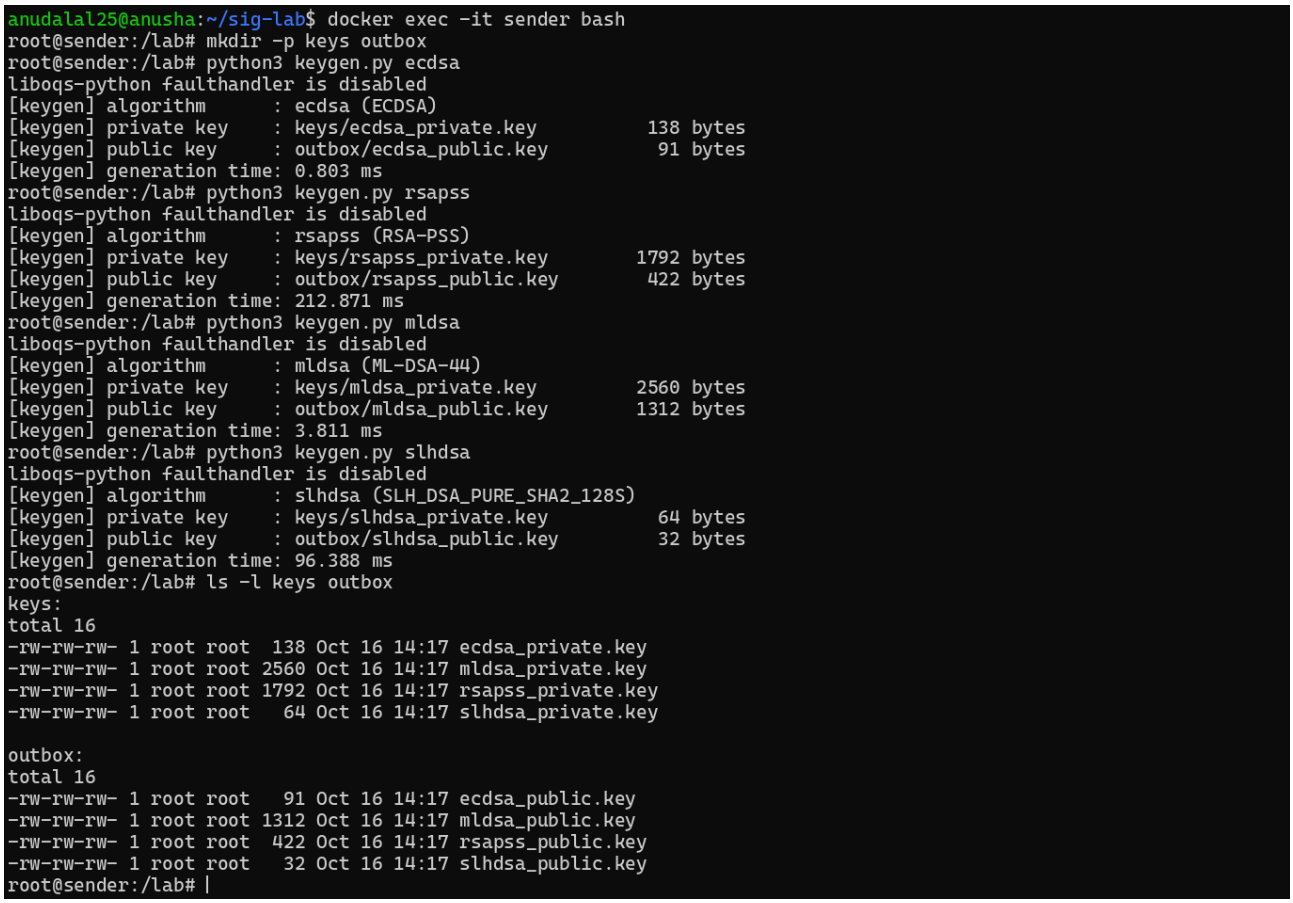
Screenshot 6: Step C8: execution environment (CPU, RAM, OS, Docker, cryptography and liboqs versions) for Table 1



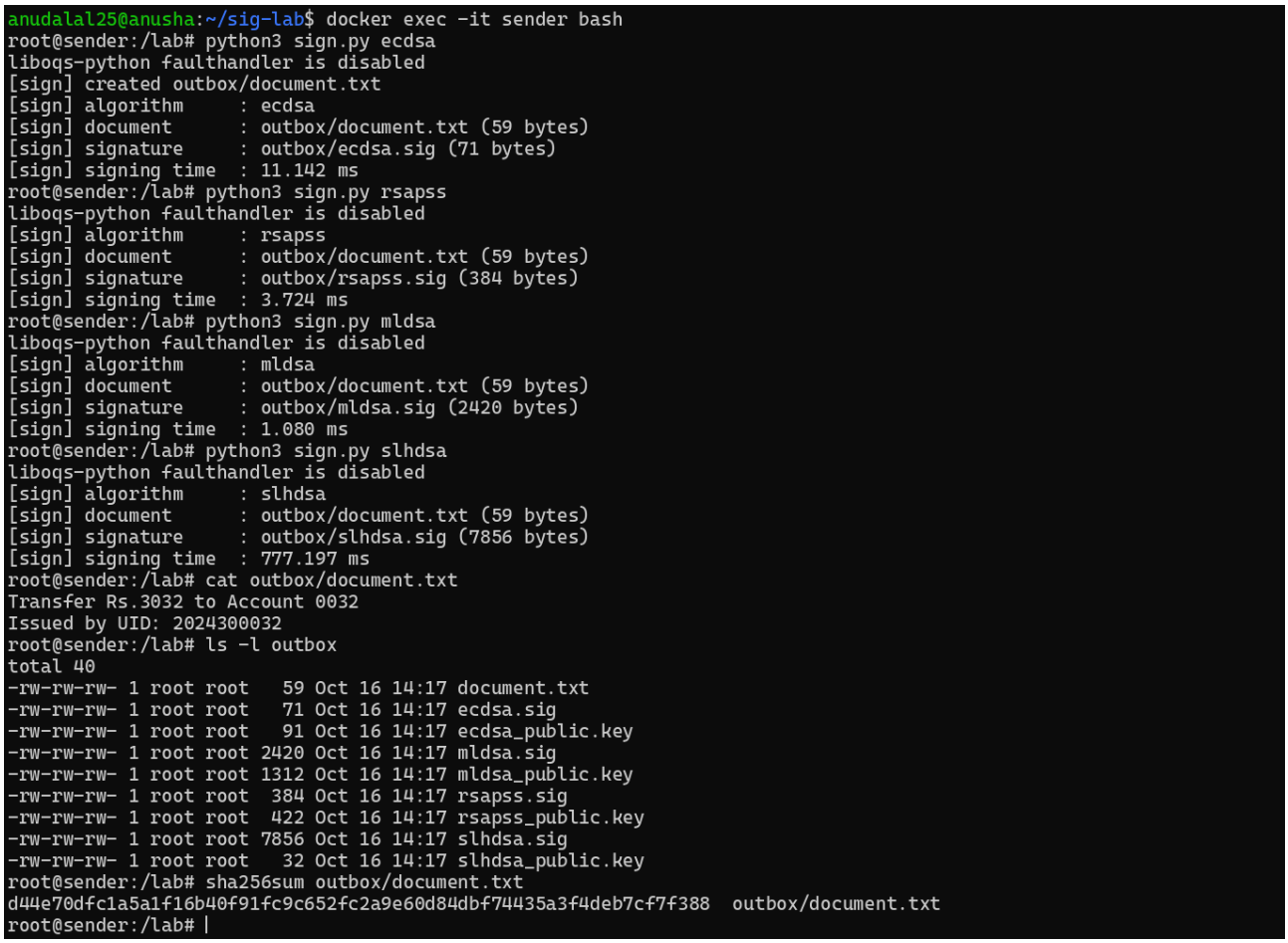
Screenshot 7: Step D1: exact ML-DSA / SLH-DSA names in liboqs (ML-DSA-44, SLH_DSA_PURE_SHA2_128S)

```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# python3 -c "import oqs; print([m for m in oqs.get_enabled_sig_mechanisms() if 'ML-DSA' in m or 'SL
H' in m])"
liboqs-python faulthandler is disabled
['ML-DSA-44', 'ML-DSA-65', 'ML-DSA-87', 'SLH_DSA_PURE_SHA2_128S', 'SLH_DSA_PURE_SHA2_128F', 'SLH_DSA_PURE_SHA2_192S'
, 'SLH_DSA_PURE_SHA2_192F', 'SLH_DSA_PURE_SHA2_256S', 'SLH_DSA_PURE_SHA2_256F', 'SLH_DSA_PURE_SHAKE_128S', 'SLH_DSA_
PURE_SHAKE_128F', 'SLH_DSA_PURE_SHAKE_192S', 'SLH_DSA_PURE_SHAKE_192F', 'SLH_DSA_PURE_SHAKE_256S', 'SLH_DSA_PURE_SHA
KE_256F', 'SLH_DSA_SHA2_224_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_224_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_224_PREHASH_SHA2
_192S', 'SLH_DSA_SHA2_224_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_224_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_224_PREHASH_SHA2_2
56F', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_
192S', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_224_PREHASH_SHAKE
_256F', 'SLH_DSA_SHA2_256_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_256_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_256_PREHASH_SHA2_1
92S', 'SLH_DSA_SHA2_256_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_256_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_256_PREHASH_SHA2_256
F', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_19
2S', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_2
56F', 'SLH_DSA_SHA2_384_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_384_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_384_PREHASH_SHA2_192
S', 'SLH_DSA_SHA2_384_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_384_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_384_PREHASH_SHA2_256F'
, 'SLH_DSA_SHA2_384_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_192S
', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_256
F', 'SLH_DSA_SHA2_512_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_512_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_512_PREHASH_SHA2_192S'
, 'SLH_DSA_SHA2_512_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_512_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_512_PREHASH_SHA2_256F',
'SLH_DSA_SHA2_512_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_192S',
 'SLH_DSA_SHA2_512_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_256F'
, 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_512_224_PREHASH_
SHA2_192S', 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_512_22
4_PREHASH_SHA2_256F', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_128F', 'SLH_DSA
_SHA2_512_224_PREHASH_SHAKE_192S', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_25
6S', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_256F', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_512_256_PREH
ASH_SHA2_128F', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_192S', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_51
2_256_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_256F', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_128S', 'SLH_
DSA_SHA2_512_256_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_192S', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE
_192F', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_256F', 'SLH_DSA_SHA3_224_PREH
ASH_SHA2_128S', 'SLH_DSA_SHA3_224_PREHASH_SHA2_128F', 'SLH_DSA_SHA3_224_PREHASH_SHA2_192S', 'SLH_DSA_SHA3_224_PREHAS
H_SHA2_192F', 'SLH_DSA_SHA3_224_PREHASH_SHA2_256S', 'SLH_DSA_SHA3_224_PREHASH_SHA2_256F', 'SLH_DSA_SHA3_224_PREHASH_
SHAKE_128S', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_128F', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_192S', 'SLH_DSA_SHA3_224_PREHASH
_SHAKE_192F', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_256S', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_256F', 'SLH_DSA_SHA3_256_PREHAS
H_SHA2_128S', 'SLH_DSA_SHA3_256_PREHASH_SHA2_128F', 'SLH_DSA_SHA3_256_PREHASH_SHA2_192S', 'SLH_DSA_SHA3_256_PREHASH_
SHA2_192F', 'SLH_DSA_SHA3_256_PREHASH_SHA2_256S', 'SLH_DSA_SHA3_256_PREHASH_SHA2_256F', 'SLH_DSA_SHA3_256_PREHASH_SH
AKE_128S', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_128F', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_192S', 'SLH_DSA_SHA3_256_PREHASH_S
HAKE_192F', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_256S', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_256F', 'SLH_DSA_SHA3_384_PREHASH_
SHA2_128S', 'SLH_DSA_SHA3_384_PREHASH_SHA2_128F', 'SLH_DSA_SHA3_384_PREHASH_SHA2_192S', 'SLH_DSA_SHA3_384_PREHASH_SH
A2_192F', 'SLH_DSA_SHA3_384_PREHASH_SHA2_256S', 'SLH_DSA_SHA3_384_PREHASH_SHA2_256F', 'SLH_DSA_SHA3_384_PREHASH_SHAK
E_128S', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_128F', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_192S', 'SLH_DSA_SHA3_384_PREHASH_SHA
KE_192F', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_256S', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_256F', 'SLH_DSA_SHA3_512_PREHASH_SH
A2_128S', 'SLH_DSA_SHA3_512_PREHASH_SHA2_128F', 'SLH_DSA_SHA3_512_PREHASH_SHA2_192S', 'SLH_DSA_SHA3_512_PREHASH_SHA2
_192F', 'SLH_DSA_SHA3_512_PREHASH_SHA2_256S', 'SLH_DSA_SHA3_512_PREHASH_SHA2_256F', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_
128S', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_128F', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_192S', 'SLH_DSA_SHA3_512_PREHASH_SHAKE
_192F', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_256S', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_256F', 'SLH_DSA_SHAKE_128_PREHASH_SHA
2_128S', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_128F', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_192S', 'SLH_DSA_SHAKE_128_PREHASH_SH
A2_192F', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_256S', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_256F', 'SLH_DSA_SHAKE_128_PREHASH_S
HAKE_128S', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_128F', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_192S', 'SLH_DSA_SHAKE_128_PREHA
SH_SHAKE_192F', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_256S', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_256F', 'SLH_DSA_SHAKE_256_P
REHASH_SHA2_128S', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_128F', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_192S', 'SLH_DSA_SHAKE_256_
PREHASH_SHA2_192F', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_256S', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_256F', 'SLH_DSA_SHAKE_256
_PREHASH_SHAKE_128S', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_128F', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_192S', 'SLH_DSA_SHAKE
_256_PREHASH_SHAKE_192F', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_256S', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_256F']
root@sender:/lab#
```

Screenshot 8: Step D3: keygen.py generates the ECDSA, RSA-PSS, ML-DSA and SLH-DSA key pairs and prints their sizes



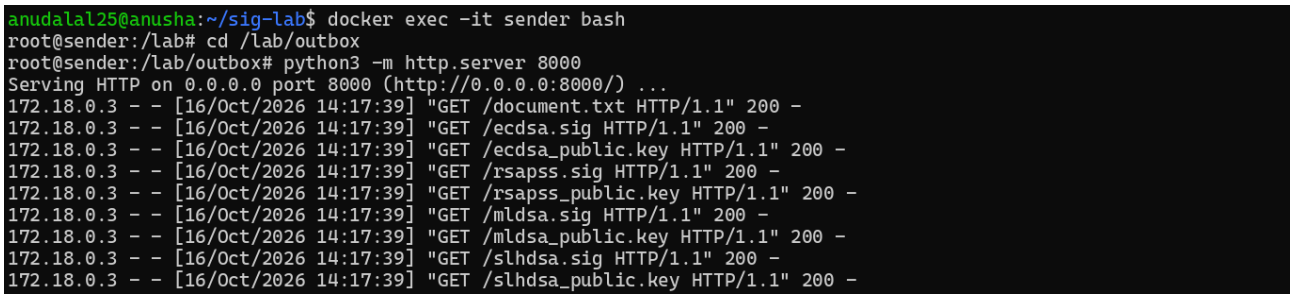
Screenshot 9: Part E: sign.py creates the UID document and signs it with all four algorithms; document SHA-256 noted



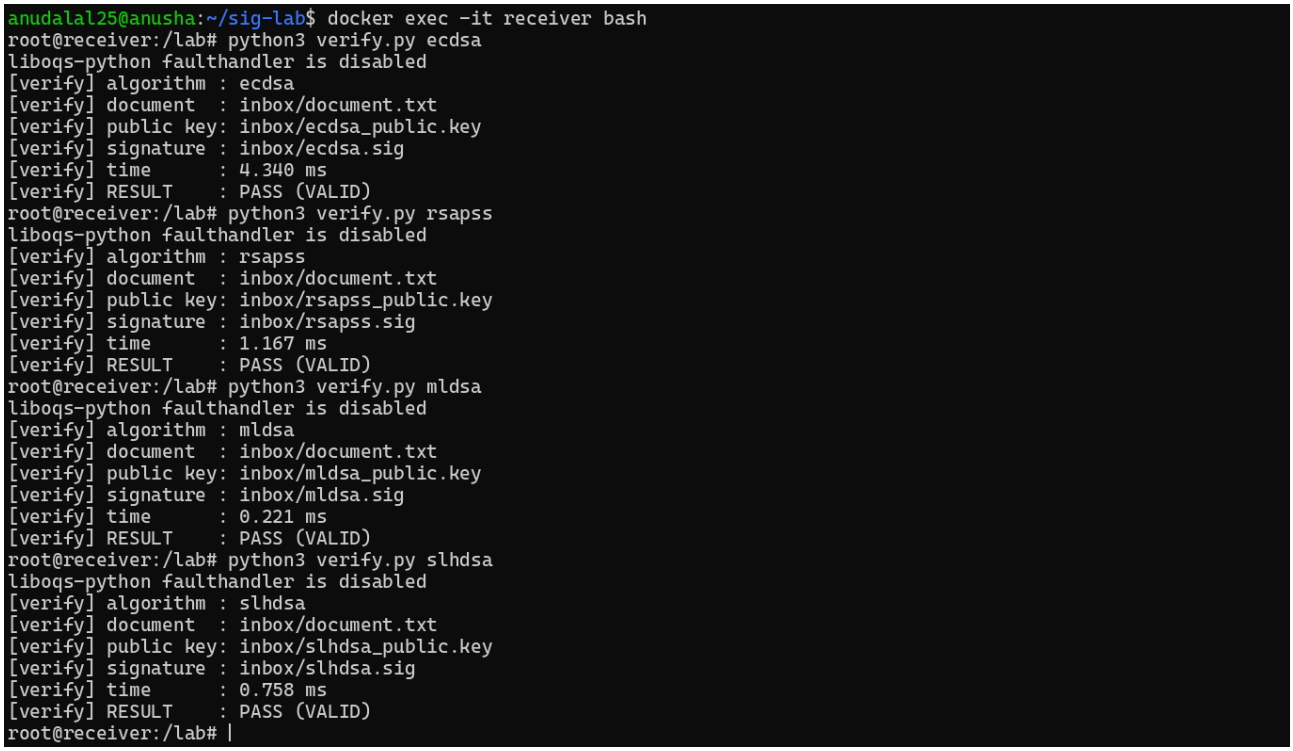
Screenshot 10: Part F: receiver downloads document, signatures and public keys from http://sender:8000; SHA-256 matches

```
anudalal25@anusha:~/sig-lab$ docker exec -it receiver bash
root@receiver:/lab# mkdir -p /lab/inbox && cd /lab/inbox && curl -O http://sender:8000/document.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100    59  100    59    0     0   5880      0 --:--:-- --:--:-- --:--:--  6555
root@receiver:/lab/inbox# for a in ecdsa rsapss mldsa slhdsa; do curl -sO http://sender:8000/${a}.sig; curl -sO http
://sender:8000/${a}_public.key; done
root@receiver:/lab/inbox# ls -l /lab/inbox
total 40
-rw-rw-rw- 1 root root   59 Oct 16 14:17 document.txt
-rw-rw-rw- 1 root root   71 Oct 16 14:17 ecdsa.sig
-rw-rw-rw- 1 root root   91 Oct 16 14:17 ecdsa_public.key
-rw-rw-rw- 1 root root 2420 Oct 16 14:17 mldsa.sig
-rw-rw-rw- 1 root root 1312 Oct 16 14:17 mldsa_public.key
-rw-rw-rw- 1 root root  384 Oct 16 14:17 rsapss.sig
-rw-rw-rw- 1 root root  422 Oct 16 14:17 rsapss_public.key
-rw-rw-rw- 1 root root 7856 Oct 16 14:17 slhdsa.sig
-rw-rw-rw- 1 root root   32 Oct 16 14:17 slhdsa_public.key
root@receiver:/lab/inbox# sha256sum /lab/inbox/document.txt
d44e70dfc1a5a1f16b40f91fc9c652fc2a9e60d84dbf74435a3f4deb7cf7f388  /lab/inbox/document.txt
root@receiver:/lab/inbox#
```

Screenshot 11: Part F: sender's HTTP server log showing every request coming from the receiver's IP (172.18.0.3)



Screenshot 12: Part G: verify.py on the receiver prints PASS for all four algorithms



Screenshot 13: Part G: key, signature and document sizes (stat -c %s) on the sender and bytes received for Table 2

```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# for a in ecdsa rsapss mldsa slhdsa; do echo "$a public=$(stat -c %s outbox/${a}_public.key) privat
e=$(stat -c %s keys/${a}_private.key) sig=$(stat -c %s outbox/${a}.sig) doc=$(stat -c %s outbox/document.txt)"; done
ecdsa public=91 private=138 sig=71 doc=59
rsapss public=422 private=1792 sig=384 doc=59
mldsa public=1312 private=2560 sig=2420 doc=59
slhdsa public=32 private=64 sig=7856 doc=59
root@sender:/lab# exit
exit
anudalal25@anusha:~/sig-lab$ docker exec -it receiver bash
root@receiver:/lab# for a in ecdsa rsapss mldsa slhdsa; do echo "$a received: doc+sig+pk = $(( $(stat -c %s inbox/do
cument.txt) + $(stat -c %s inbox/${a}.sig) + $(stat -c %s inbox/${a}_public.key) )) bytes"; done
ecdsa received: doc+sig+pk = 221 bytes
rsapss received: doc+sig+pk = 865 bytes
mldsa received: doc+sig+pk = 3791 bytes
slhdsa received: doc+sig+pk = 7947 bytes
root@receiver:/lab#
```

Screenshot 14: Part H, Tests 1-2: one character of the document changed; original passes, modified document FAILS

```
anudalal25@anusha:~/sig-lab$ docker exec -it receiver bash
root@receiver:/lab# python3 tamper.py doc inbox/document.txt
[tamper] file      : inbox/document.txt (59 bytes)
[tamper] index     : UID mod 59 = 2
[tamper] byte      : 0x61 -> 0x60 (bit 0 flipped)
[tamper] character : 'a' -> '`'
[tamper] original text:
Transfer Rs.3032 to Account 0032
Issued by UID: 2024300032
[tamper] modified text:
Tr`nsfer Rs.3032 to Account 0032
Issued by UID: 2024300032
[tamper] written   : inbox/document.txt.tampered
root@receiver:/lab# python3 verify.py ecdsa inbox/document.txt inbox/ecdsa_public.key inbox/ecdsa.sig    # Test 1: o
riginal
liboqs-python faulthandler is disabled
[verify] algorithm : ecdsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/ecdsa_public.key
[verify] signature : inbox/ecdsa.sig
[verify] time      : 4.987 ms
[verify] RESULT    : PASS (VALID)
root@receiver:/lab# python3 verify.py rsapss inbox/document.txt inbox/rsapss_public.key inbox/rsapss.sig    # Test 1
: original
liboqs-python faulthandler is disabled
[verify] algorithm : rsapss
[verify] document  : inbox/document.txt
[verify] public key: inbox/rsapss_public.key
[verify] signature : inbox/rsapss.sig
[verify] time      : 1.295 ms
[verify] RESULT    : PASS (VALID)
root@receiver:/lab# python3 verify.py mldsa inbox/document.txt inbox/mldsa_public.key inbox/mldsa.sig    # Test 1: o
riginal
liboqs-python faulthandler is disabled
[verify] algorithm : mldsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/mldsa_public.key
[verify] signature : inbox/mldsa.sig
[verify] time      : 0.263 ms
[verify] RESULT    : PASS (VALID)
root@receiver:/lab# python3 verify.py slhdsa inbox/document.txt inbox/slhdsa_public.key inbox/slhdsa.sig    # Test 1
: original
liboqs-python faulthandler is disabled
[verify] algorithm : slhdsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/slhdsa_public.key
[verify] signature : inbox/slhdsa.sig
[verify] time      : 0.872 ms
[verify] RESULT    : PASS (VALID)
root@receiver:/lab# python3 verify.py ecdsa inbox/document.txt.tampered    # Test 2: modified document
liboqs-python faulthandler is disabled
[verify] algorithm : ecdsa
[verify] document  : inbox/document.txt.tampered
[verify] public key: inbox/ecdsa_public.key
[verify] signature : inbox/ecdsa.sig
[verify] time      : 4.088 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 verify.py rsapss inbox/document.txt.tampered    # Test 2: modified document
liboqs-python faulthandler is disabled
[verify] algorithm : rsapss
[verify] document  : inbox/document.txt.tampered
[verify] public key: inbox/rsapss_public.key
[verify] signature : inbox/rsapss.sig
[verify] time      : 1.755 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 verify.py mldsa inbox/document.txt.tampered    # Test 2: modified document
liboqs-python faulthandler is disabled
[verify] algorithm : mldsa
[verify] document  : inbox/document.txt.tampered
[verify] public key: inbox/mldsa_public.key
[verify] signature : inbox/mldsa.sig
[verify] time      : 0.255 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 verify.py slhdsa inbox/document.txt.tampered    # Test 2: modified document
liboqs-python faulthandler is disabled
[verify] algorithm : slhdsa
[verify] document  : inbox/document.txt.tampered
[verify] public key: inbox/slhdsa_public.key
[verify] signature : inbox/slhdsa.sig
[verify] time      : 0.766 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab#
```

Screenshot 15: Part H, Test 3: one bit of each signature flipped at index UID mod length; all four FAIL

```
anudalal25@anusha:~/sig-lab$ docker exec -it receiver bash
root@receiver:/lab# python3 tamper.py sig inbox/ecdsa.sig
[tamper] file      : inbox/ecdsa.sig (71 bytes)
[tamper] index     : UID mod 71 = 4
[tamper] byte      : 0x00 -> 0x01 (bit 0 flipped)
[tamper] written   : inbox/ecdsa.sig.tampered
root@receiver:/lab# python3 verify.py ecdsa inbox/document.txt inbox/ecdsa_public.key inbox/ecdsa.sig.tampered    #
Test 3: modified signature
liboqs-python faulthandler is disabled
[verify] algorithm : ecdsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/ecdsa_public.key
[verify] signature : inbox/ecdsa.sig.tampered
[verify] time      : 3.898 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 tamper.py sig inbox/rsapss.sig
[tamper] file      : inbox/rsapss.sig (384 bytes)
[tamper] index     : UID mod 384 = 256
[tamper] byte      : 0x9b -> 0x9a (bit 0 flipped)
[tamper] written   : inbox/rsapss.sig.tampered
root@receiver:/lab# python3 verify.py rsapss inbox/document.txt inbox/rsapss_public.key inbox/rsapss.sig.tampered
 # Test 3: modified signature
liboqs-python faulthandler is disabled
[verify] algorithm : rsapss
[verify] document  : inbox/document.txt
[verify] public key: inbox/rsapss_public.key
[verify] signature : inbox/rsapss.sig.tampered
[verify] time      : 1.162 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 tamper.py sig inbox/mldsa.sig
[tamper] file      : inbox/mldsa.sig (2420 bytes)
[tamper] index     : UID mod 2420 = 1492
[tamper] byte      : 0x09 -> 0x08 (bit 0 flipped)
[tamper] written   : inbox/mldsa.sig.tampered
root@receiver:/lab# python3 verify.py mldsa inbox/document.txt inbox/mldsa_public.key inbox/mldsa.sig.tampered    #
Test 3: modified signature
liboqs-python faulthandler is disabled
[verify] algorithm : mldsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/mldsa_public.key
[verify] signature : inbox/mldsa.sig.tampered
[verify] time      : 0.178 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 tamper.py sig inbox/slhdsa.sig
[tamper] file      : inbox/slhdsa.sig (7856 bytes)
[tamper] index     : UID mod 7856 = 5232
[tamper] byte      : 0x53 -> 0x52 (bit 0 flipped)
[tamper] written   : inbox/slhdsa.sig.tampered
root@receiver:/lab# python3 verify.py slhdsa inbox/document.txt inbox/slhdsa_public.key inbox/slhdsa.sig.tampered
 # Test 3: modified signature
liboqs-python faulthandler is disabled
[verify] algorithm : slhdsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/slhdsa_public.key
[verify] signature : inbox/slhdsa.sig.tampered
[verify] time      : 0.762 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab#
```

Screenshot 16: Part H, Test 4: receiver makes its own key pairs (independent ECDSA key) - verification with the wrong public key FAILS for all four

```
anudalal25@anusha:~/sig-lab$ docker exec -it receiver bash
root@receiver:/lab# mkdir -p keys outbox
root@receiver:/lab# python3 keygen.py ecdsa --independent    # Test 4: receiver makes ITS OWN key pair
liboqs-python faulthandler is disabled
[keygen] algorithm      : ecdsa (ECDSA)  [independent key from the secure RNG]
[keygen] private key    : keys/ecdsa_private.key          138 bytes
[keygen] public key     : outbox/ecdsa_public.key          91 bytes
[keygen] generation time: 35.439 ms
root@receiver:/lab# python3 verify.py ecdsa inbox/document.txt outbox/ecdsa_public.key inbox/ecdsa.sig    # Test 4:
wrong public key
liboqs-python faulthandler is disabled
[verify] algorithm : ecdsa
[verify] document  : inbox/document.txt
[verify] public key: outbox/ecdsa_public.key
[verify] signature : inbox/ecdsa.sig
[verify] time      : 11.863 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 keygen.py rsapss    # Test 4: receiver makes ITS OWN key pair
liboqs-python faulthandler is disabled
[keygen] algorithm      : rsapss (RSA-PSS)
[keygen] private key    : keys/rsapss_private.key        1794 bytes
[keygen] public key     : outbox/rsapss_public.key        422 bytes
[keygen] generation time: 31.473 ms
root@receiver:/lab# python3 verify.py rsapss inbox/document.txt outbox/rsapss_public.key inbox/rsapss.sig    # Test
4: wrong public key
liboqs-python faulthandler is disabled
[verify] algorithm : rsapss
[verify] document  : inbox/document.txt
[verify] public key: outbox/rsapss_public.key
[verify] signature : inbox/rsapss.sig
[verify] time      : 3.119 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 keygen.py mldsa    # Test 4: receiver makes ITS OWN key pair
liboqs-python faulthandler is disabled
[keygen] algorithm      : mldsa (ML-DSA-44)
[keygen] private key    : keys/mldsa_private.key         2560 bytes
[keygen] public key     : outbox/mldsa_public.key        1312 bytes
[keygen] generation time: 94.715 ms
root@receiver:/lab# python3 verify.py mldsa inbox/document.txt outbox/mldsa_public.key inbox/mldsa.sig    # Test 4:
wrong public key
liboqs-python faulthandler is disabled
[verify] algorithm : mldsa
[verify] document  : inbox/document.txt
[verify] public key: outbox/mldsa_public.key
[verify] signature : inbox/mldsa.sig
[verify] time      : 0.464 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 keygen.py slhdsa    # Test 4: receiver makes ITS OWN key pair
liboqs-python faulthandler is disabled
[keygen] algorithm      : slhdsa (SLH_DSA_PURE_SHA2_128S)
[keygen] private key    : keys/slhdsa_private.key          64 bytes
[keygen] public key     : outbox/slhdsa_public.key         32 bytes
[keygen] generation time: 149.479 ms
root@receiver:/lab# python3 verify.py slhdsa inbox/document.txt outbox/slhdsa_public.key inbox/slhdsa.sig    # Test
4: wrong public key
liboqs-python faulthandler is disabled
[verify] algorithm : slhdsa
[verify] document  : inbox/document.txt
[verify] public key: outbox/slhdsa_public.key
[verify] signature : inbox/slhdsa.sig
[verify] time      : 0.963 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# for a in ecdsa rsapss mldsa slhdsa; do cmp -s inbox/${a}_public.key outbox/${a}_public.key && ec
ho "$a: receiver key IDENTICAL to sender key (not a wrong key!)" || echo "$a: receiver key differs from sender key";
 done
ecdsa: receiver key differs from sender key
rsapss: receiver key differs from sender key
mldsa: receiver key differs from sender key
slhdsa: receiver key differs from sender key
root@receiver:/lab#
```

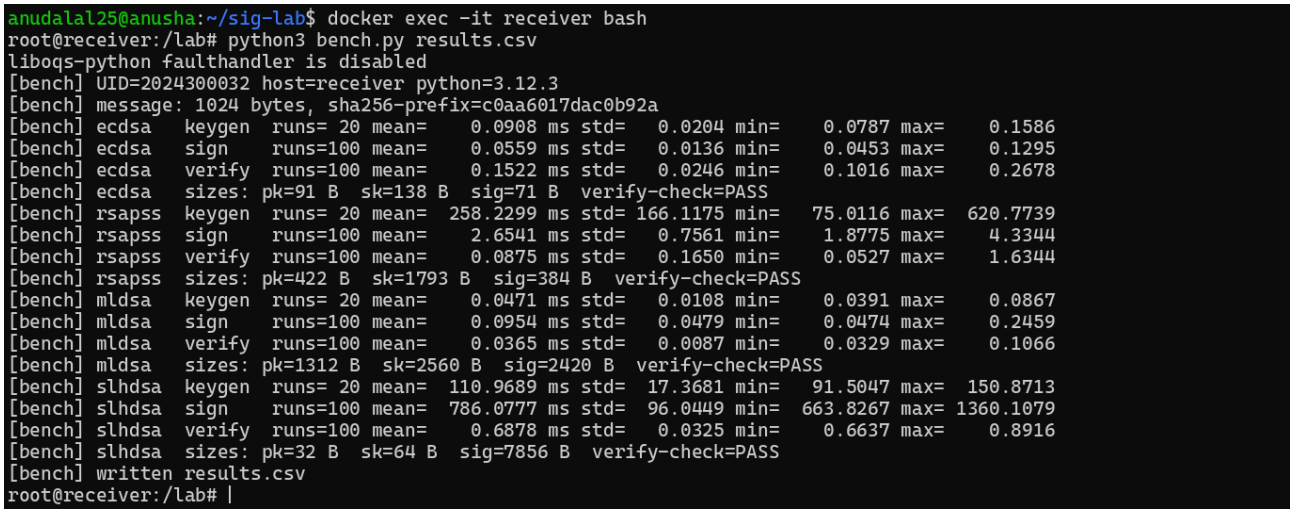
Screenshot 17: Part I: bench.py on the sender, run three times (results_run1/2/3.csv)

```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# python3 bench.py results_run1.csv
liboqs-python faulthandler is disabled
[bench] UID=2024300032 host=sender python=3.12.3
[bench] message: 1024 bytes, sha256-prefix=c0aa6017dac0b92a
[bench] ecdsa   keygen  runs= 20 mean=    0.0531 ms std=   0.0124 min=    0.0465 max=    0.0936
[bench] ecdsa   sign    runs=100 mean=    0.0324 ms std=   0.0046 min=    0.0307 max=    0.0624
[bench] ecdsa   verify  runs=100 mean=    0.0827 ms std=   0.0178 min=    0.0746 max=    0.2070
[bench] ecdsa   sizes: pk=91 B  sk=138 B  sig=70 B  verify-check=PASS
[bench] rsapss  keygen  runs= 20 mean=  242.5821 ms std= 133.5144 min=   60.2655 max=  508.3913
[bench] rsapss  sign    runs=100 mean=    3.1595 ms std=   0.8135 min=    1.8585 max=    5.1465
[bench] rsapss  verify  runs=100 mean=    0.1090 ms std=   0.0227 min=    0.0915 max=    0.2389
[bench] rsapss  sizes: pk=422 B  sk=1792 B  sig=384 B  verify-check=PASS
[bench] mldsa   keygen  runs= 20 mean=    0.0800 ms std=   0.0148 min=    0.0709 max=    0.1213
[bench] mldsa   sign    runs=100 mean=    0.1764 ms std=   0.0932 min=    0.0686 max=    0.5427
[bench] mldsa   verify  runs=100 mean=    0.0636 ms std=   0.0129 min=    0.0478 max=    0.1204
[bench] mldsa   sizes: pk=1312 B  sk=2560 B  sig=2420 B  verify-check=PASS
[bench] slhdsa  keygen  runs= 20 mean=   98.5066 ms std=  12.4914 min=   87.1956 max=  125.0923
[bench] slhdsa  sign    runs=100 mean=  850.8062 ms std= 117.7786 min=  695.9288 max= 1421.5686
[bench] slhdsa  verify  runs=100 mean=    1.0275 ms std=   0.2206 min=    0.6399 max=    1.4493
[bench] slhdsa  sizes: pk=32 B  sk=64 B  sig=7856 B  verify-check=PASS
[bench] written results_run1.csv
root@sender:/lab# python3 bench.py results_run2.csv
liboqs-python faulthandler is disabled
[bench] UID=2024300032 host=sender python=3.12.3
[bench] message: 1024 bytes, sha256-prefix=c0aa6017dac0b92a
[bench] ecdsa   keygen  runs= 20 mean=    0.0765 ms std=   0.0187 min=    0.0637 max=    0.1487
[bench] ecdsa   sign    runs=100 mean=    0.0558 ms std=   0.0225 min=    0.0389 max=    0.1707
[bench] ecdsa   verify  runs=100 mean=    0.1291 ms std=   0.0261 min=    0.1009 max=    0.2590
[bench] ecdsa   sizes: pk=91 B  sk=138 B  sig=71 B  verify-check=PASS
[bench] rsapss  keygen  runs= 20 mean=  215.7596 ms std= 172.9181 min=   34.3285 max=  663.4801
[bench] rsapss  sign    runs=100 mean=    2.2231 ms std=   0.6104 min=    1.8540 max=    6.9156
[bench] rsapss  verify  runs=100 mean=    0.0589 ms std=   0.0108 min=    0.0524 max=    0.1047
[bench] rsapss  sizes: pk=422 B  sk=1793 B  sig=384 B  verify-check=PASS
[bench] mldsa   keygen  runs= 20 mean=    0.0454 ms std=   0.0099 min=    0.0395 max=    0.0843
[bench] mldsa   sign    runs=100 mean=    0.1041 ms std=   0.0572 min=    0.0481 max=    0.2921
[bench] mldsa   verify  runs=100 mean=    0.0357 ms std=   0.0067 min=    0.0331 max=    0.0799
[bench] mldsa   sizes: pk=1312 B  sk=2560 B  sig=2420 B  verify-check=PASS
[bench] slhdsa  keygen  runs= 20 mean=  103.7940 ms std=  13.2166 min=   89.0875 max=  129.4558
[bench] slhdsa  sign    runs=100 mean=  839.7676 ms std= 118.5879 min=  663.9100 max= 1226.1801
[bench] slhdsa  verify  runs=100 mean=    0.6646 ms std=   0.0175 min=    0.6427 max=    0.7716
[bench] slhdsa  sizes: pk=32 B  sk=64 B  sig=7856 B  verify-check=PASS
[bench] written results_run2.csv
root@sender:/lab# python3 bench.py results_run3.csv
liboqs-python faulthandler is disabled
[bench] UID=2024300032 host=sender python=3.12.3
[bench] message: 1024 bytes, sha256-prefix=c0aa6017dac0b92a
[bench] ecdsa   keygen  runs= 20 mean=    0.0616 ms std=   0.0203 min=    0.0508 max=    0.1227
[bench] ecdsa   sign    runs=100 mean=    0.0406 ms std=   0.0137 min=    0.0344 max=    0.1079
[bench] ecdsa   verify  runs=100 mean=    0.0957 ms std=   0.0224 min=    0.0774 max=    0.1783
[bench] ecdsa   sizes: pk=91 B  sk=138 B  sig=72 B  verify-check=PASS
[bench] rsapss  keygen  runs= 20 mean=  214.5786 ms std= 160.9760 min=   46.7627 max=  591.4313
[bench] rsapss  sign    runs=100 mean=    2.4925 ms std=   0.8361 min=    1.8673 max=    6.8476
[bench] rsapss  verify  runs=100 mean=    0.0592 ms std=   0.0175 min=    0.0528 max=    0.1997
[bench] rsapss  sizes: pk=422 B  sk=1792 B  sig=384 B  verify-check=PASS
[bench] mldsa   keygen  runs= 20 mean=    0.0460 ms std=   0.0115 min=    0.0397 max=    0.0881
[bench] mldsa   sign    runs=100 mean=    0.1051 ms std=   0.0583 min=    0.0483 max=    0.3094
[bench] mldsa   verify  runs=100 mean=    0.0384 ms std=   0.0256 min=    0.0329 max=    0.2759
[bench] mldsa   sizes: pk=1312 B  sk=2560 B  sig=2420 B  verify-check=PASS
[bench] slhdsa  keygen  runs= 20 mean=  113.1117 ms std=  21.1980 min=   93.4412 max=  159.2152
[bench] slhdsa  sign    runs=100 mean=  809.1426 ms std= 102.8066 min=  666.6040 max= 1193.2926
[bench] slhdsa  verify  runs=100 mean=    1.3356 ms std=   0.0252 min=    1.2719 max=    1.4094
[bench] slhdsa  sizes: pk=32 B  sk=64 B  sig=7856 B  verify-check=PASS
[bench] written results_run3.csv
root@sender:/lab#
```

Screenshot 18: Part I: the median run (run 2) is kept as results.csv on the sender

```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# for r in 1 2 3; do echo "run $r: sum of mean_ms = $(awk -F, 'NR>1{s+=$5} END{printf "%.2f", s}' re
sults_run$r.csv)"; done
run 1: sum of mean_ms = 1196.68
run 2: sum of mean_ms = 1162.71
run 3: sum of mean_ms = 1141.11
root@sender:/lab# cp results_run2.csv results.csv && cat results.csv
uid,algorithm,operation,runs,mean_ms,std_ms,min_ms,max_ms,public_key_bytes,private_key_bytes,signature_bytes
2024300032,ecdsa,keygen,20,0.0765,0.0187,0.0637,0.1487,91,138,71
2024300032,ecdsa,sign,100,0.0558,0.0225,0.0389,0.1707,91,138,71
2024300032,ecdsa,verify,100,0.1291,0.0261,0.1009,0.2590,91,138,71
2024300032,rsapss,keygen,20,215.7596,172.9181,34.3285,663.4801,422,1793,384
2024300032,rsapss,sign,100,2.2231,0.6104,1.8540,6.9156,422,1793,384
2024300032,rsapss,verify,100,0.0589,0.0108,0.0524,0.1047,422,1793,384
2024300032,mldsa,keygen,20,0.0454,0.0099,0.0395,0.0843,1312,2560,2420
2024300032,mldsa,sign,100,0.1041,0.0572,0.0481,0.2921,1312,2560,2420
2024300032,mldsa,verify,100,0.0357,0.0067,0.0331,0.0799,1312,2560,2420
2024300032,slhdsa,keygen,20,103.7940,13.2166,89.0875,129.4558,32,64,7856
2024300032,slhdsa,sign,100,839.7676,118.5879,663.9100,1226.1801,32,64,7856
2024300032,slhdsa,verify,100,0.6646,0.0175,0.6427,0.7716,32,64,7856
root@sender:/lab#
```

Screenshot 19: Part I: bench.py repeated on the receiver container (receiver results.csv)



Screenshot 20: Section 15: plot.py draws the seven graphs; docker cp copies graphs/ to the host

